

@FEUP-ASSO-2025-26 C02-T3

The Seamless Shopping Experience

Migrating from Monolithic to Microservices Architecture

Before We Begin

Problem with Monolithic Architecture

The Pricing and Promotion Engine has become a **bottleneck**, where complex event processing logic introduces **1-2 second latency** during peak hours, degrading the grab-and-go experience.

As the team scaled, increased parallel development across teams introduced coordination challenges and cross-team friction.



Before We Begin

Solution

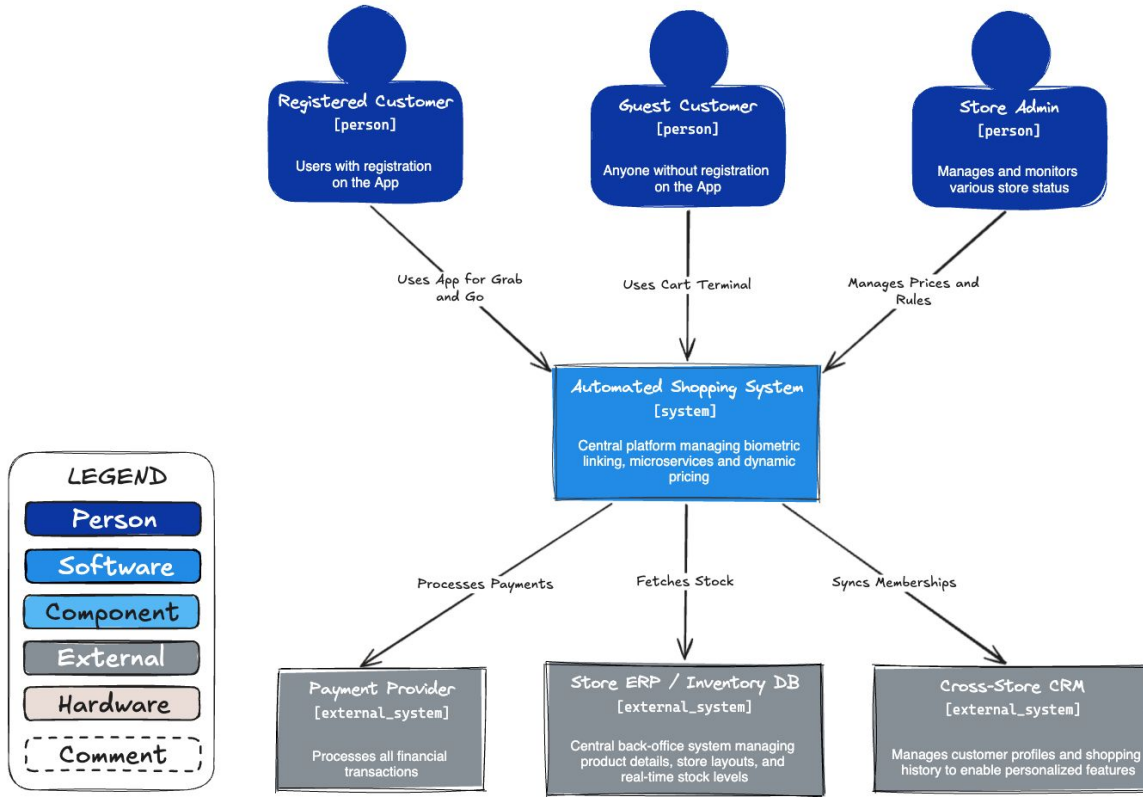
Designed a new **Domain-Driven Microservices** architecture to replace the monolithic system, decoupling the platform into highly cohesive and independently deployable services.

Provide customers a better GRAB-and-GO experience while improving developer productivity



01

System *Context*



Expanded Boundaries

Connecting the internal system to enterprise platforms

Store ERP / Inventory DB

Source of truth for physical stock and base pricing

Cross-Store CRM

Manages multi-store shopping history and loyalty tiers

Quality Attributes

High Modifiability

02

Service Boundaries

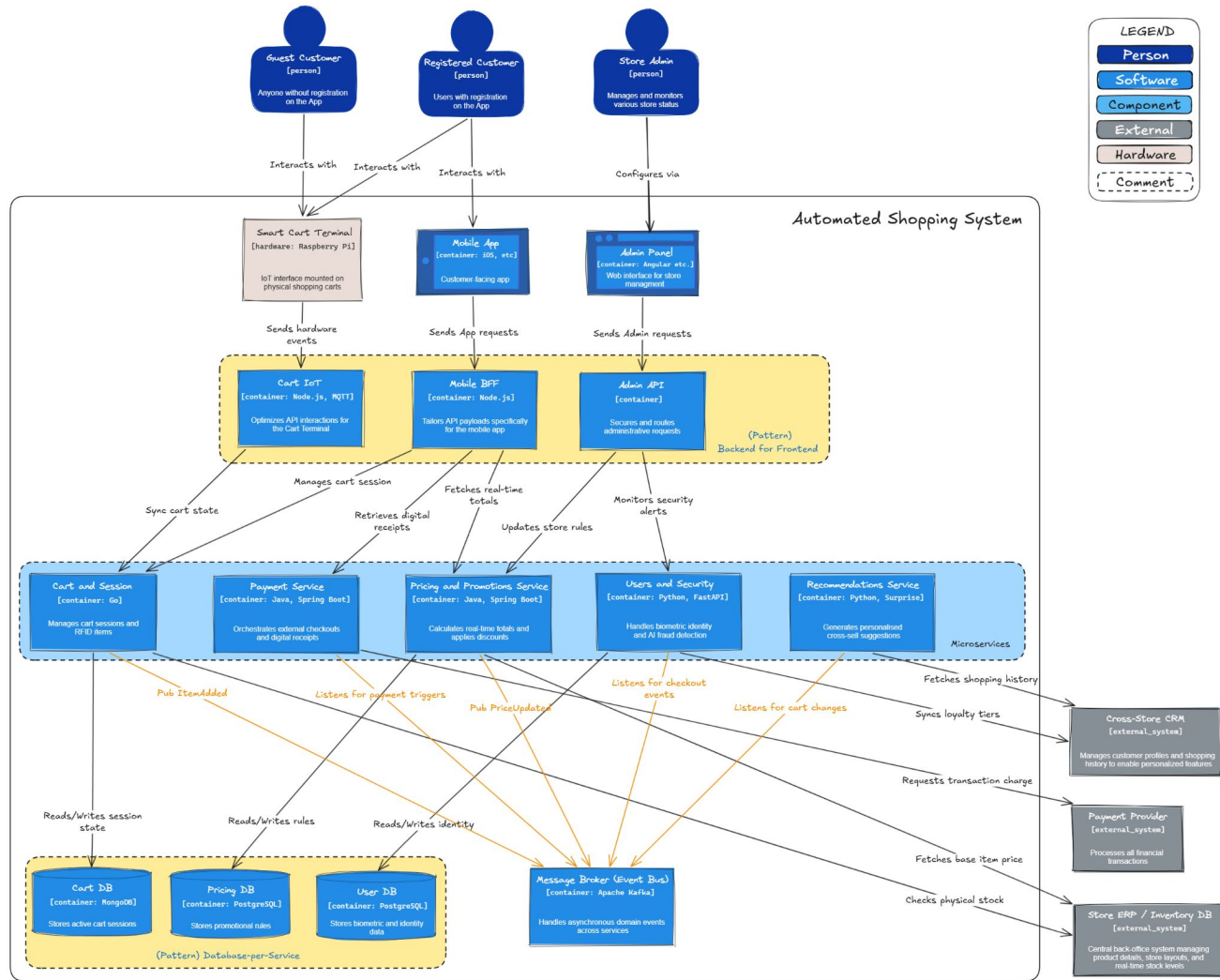
Service Boundaries

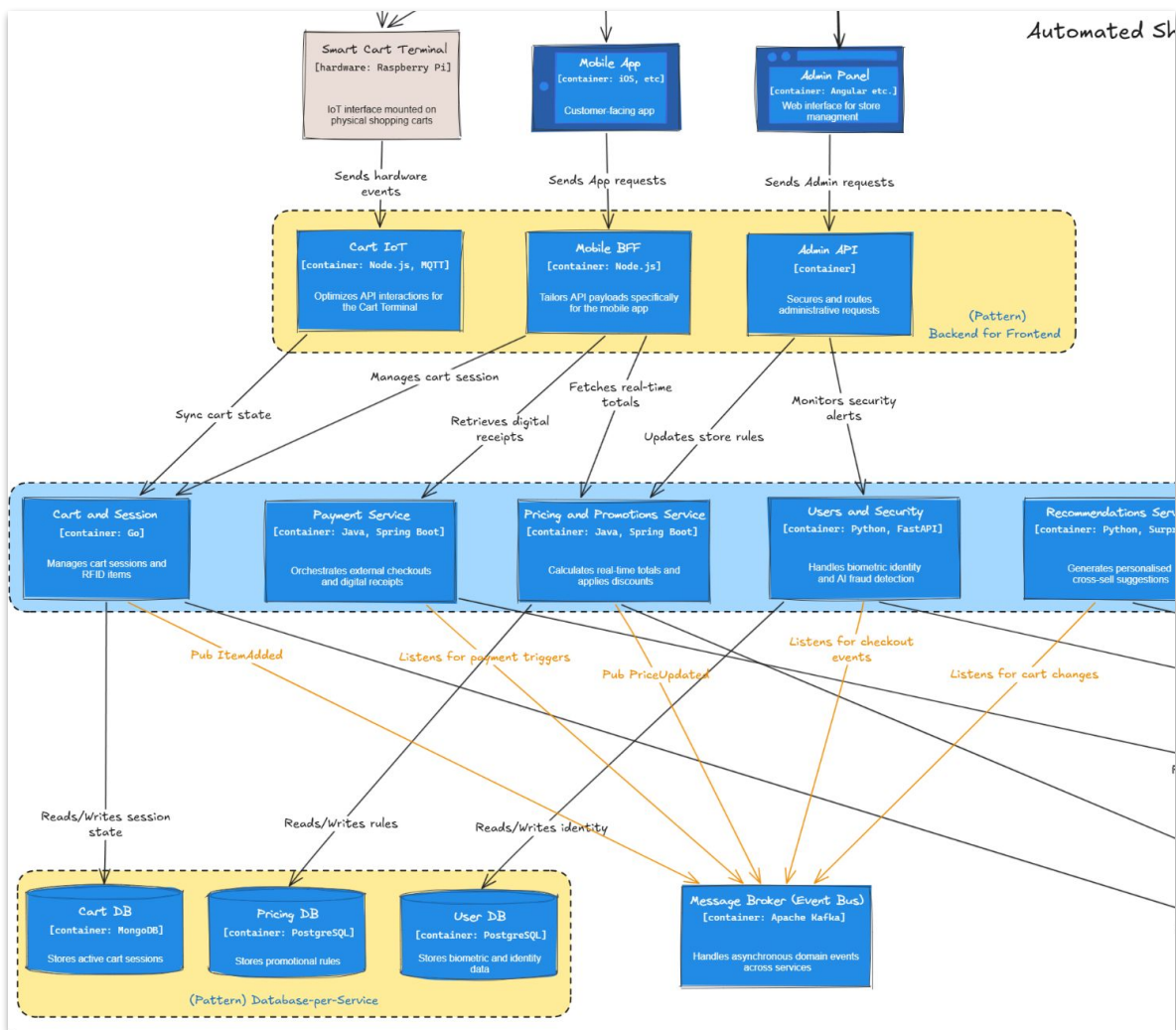
Cart & Session	Payment	Pricing & Promotions	Users & Security	Recommendations
01	02	03	04	05
Manages the active, real-time physical state of the cart	Handles the final financial transaction and digital receipt	Owns all financial calculations with discounts and flash sales.	Manages biometric identity, profiles and AI-based fraud detection	An isolated analytical engine for cross-selling and budget mode calculations

03

Cloud Adoption Patterns

C4 Level 2 Diagram





Backend for Frontend (BFF)

Dedicated APIs for Mobile App and Cart IoT, reduced Over-fetching and Under-fetching

Database-per-Service (DPS)

Decentralized data storage, NO data coupling, fault isolation

Quality Attributes

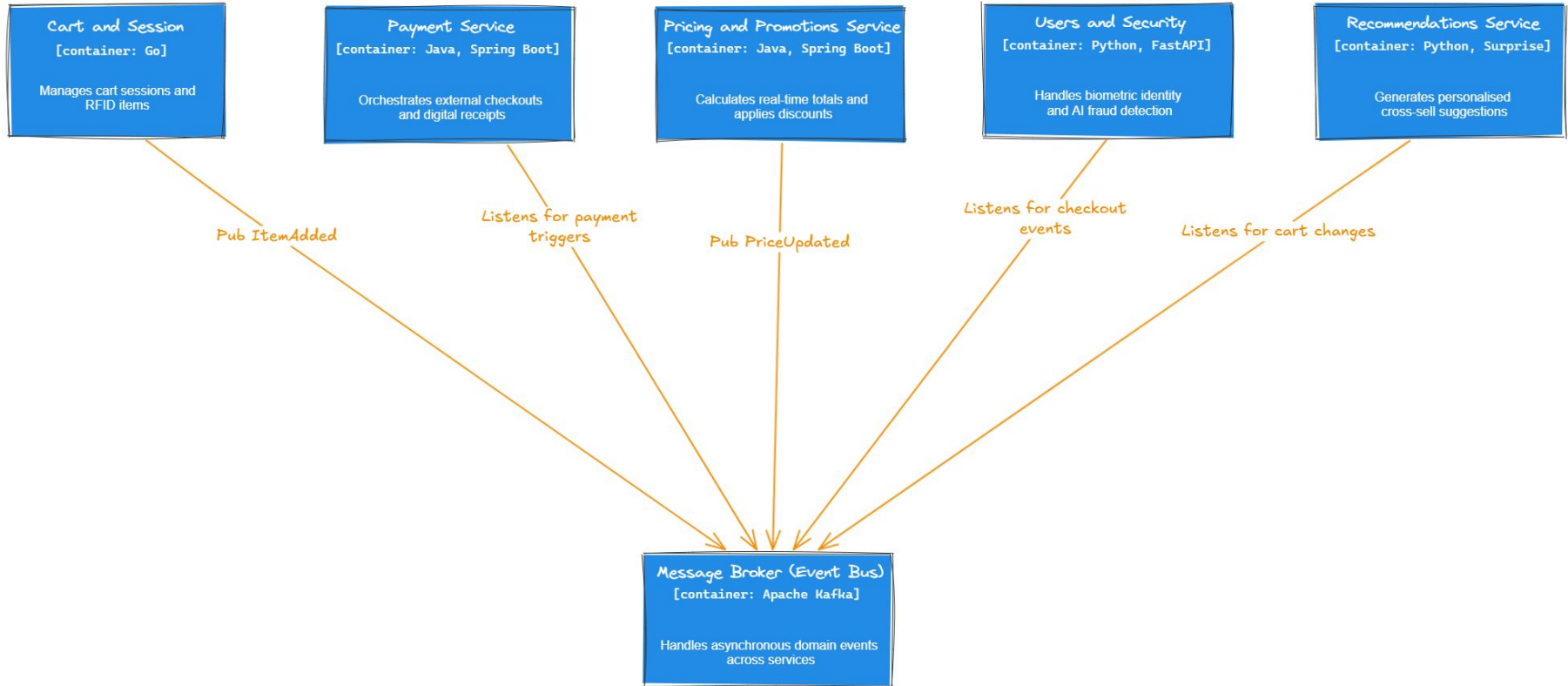
Usability and Modifiability (BFF), Availability (DPS)

Trade off complexity for system resilience

04

Event-Driven *Architecture*

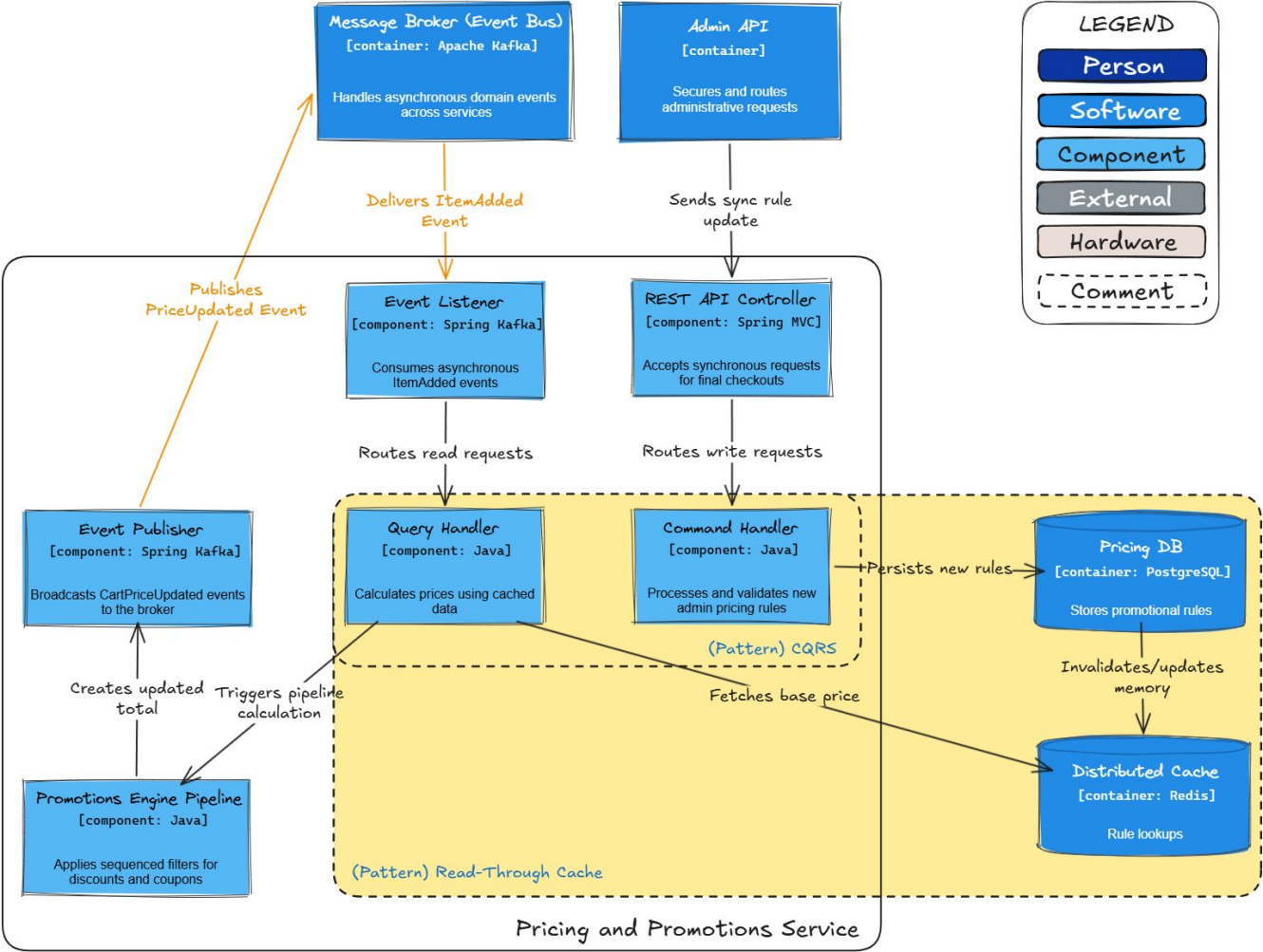
Event-Driven Architecture & Inter-Service Communication



05

Pricing & Promotions Bottleneck

C4 Level 3 Diagram



Trade-offs

Transient
Stale Data

01

If a store manager updates a flash sale, there is a microsecond window where a customer might be quoted the old price before the cache invalidate.

Operational
Complexity

02

Requires its own monitoring, replication, and failover strategy. If Redis becomes unavailable, the Query Handler has no fallback path for pricing data.

06

Migration Strategy

The Strangler Fig Strategy

Zero-Downtime Objective

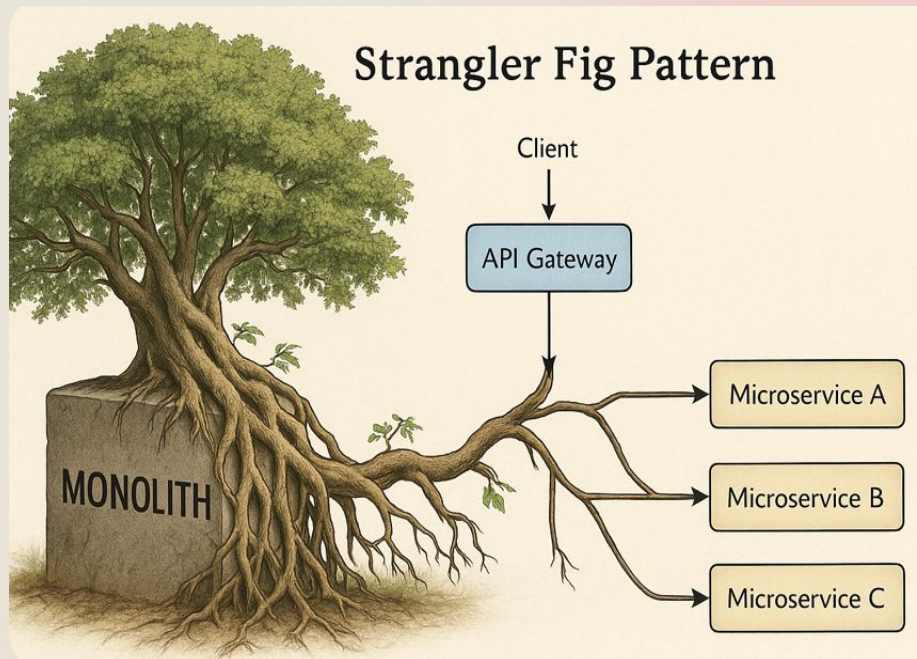
Replacing legacy core systems without interrupting high-volume retail operations.

Incremental Extraction

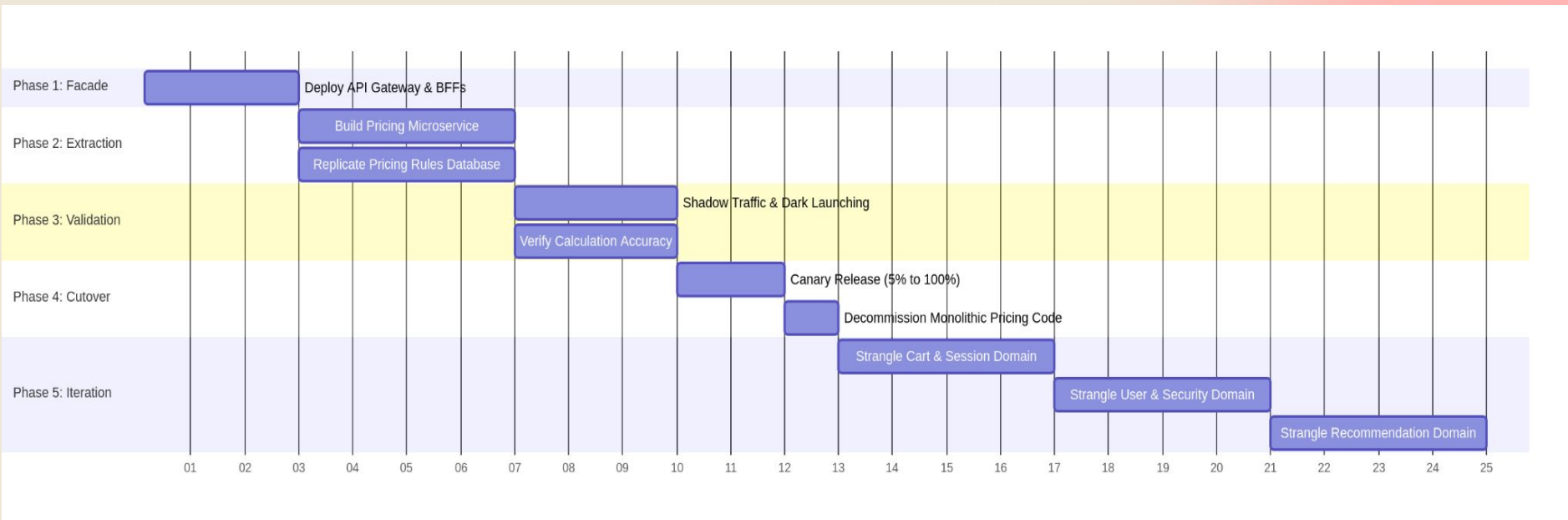
Functionalities are "strangled" out of the monolith into microservices over time.

Risk Mitigation

Explicitly avoids the "Big Bang" system replacement risk in production stores.



Phased Migration Schedule



Phase 3: Shadow Traffic

Ensures 100% calculation parity.

Phase 4: Canary Cutover

5% to 100% phased rollout.

Migration Costs and Tradeoffs



INFRASTRUCTURE OVERLAP

Maintaining concurrent legacy and modern stacks during the 25-week transition period.



SHADOW TRAFFIC OVERHEAD

Compute and database write costs effectively double during the validation phases (W8-10).



FEATURE VELOCITY

Allocating engineers to migration requires balancing extraction with new feature delivery.

These temporary costs are a necessary investment to permanently resolve our system bottlenecks and unlock long-term scalability.

07

*C*onclusion

Conclusion



DOMAIN-DRIVEN REDESIGN

Resolves monolithic latency and team development friction by decoupling into independent microservices.



MODERN CLOUD PATTERNS

Adopting EDA, BFF, and CQRS guarantees fault isolation and resilient operations.



RISK-MITIGATED MIGRATION

The Strangler Fig Pattern ensures zero-downtime, establishing a robust architecture for future growth.

Thank *You*

Designing the Future of Unemployed Cashiers